

PocketTrainer

AI Fitness Coaching App — React Native (Expo) + Node.js/Express + MongoDB

Prepared using: NIST Cybersecurity Framework (CSF) 2.0

REPORT DATE September 05, 2026

SCOPE Mobile client, Node.js/Express API,
MongoDB data layer, third-party integrations

METHOD Static/manual source code review of the
supplied repository

CLASSIFICATION CONFIDENTIAL — security-sensitive findings

1

CRITICAL

1

HIGH

6

MEDIUM

2

LOW

Table of Contents

- 1. Executive Summary
- 2. Scope & Methodology
- 3. Findings Register (Severity Overview)
- 4. Detailed Findings
- 5. NIST CSF 2.0 Function Alignment
- 6. Prioritized Remediation Roadmap
- 7. Recommendations Quick-Reference
- 8. Positive Observations
- 9. Additional Recommendations to Improve the App
- 10. Appendix — Files Reviewed

1. Executive Summary

PocketTrainer is a fitness-coaching mobile app (Expo/React Native) backed by an Express/MongoDB API deployed on Vercel, with an AI workout/chat feature (a Modal-hosted model), exercise GIF lookups, and email-based password reset via Gmail SMTP. This review covers the mobile client, backend API, data models, authentication flow, and third-party integrations as supplied.

The codebase shows several good foundations — bcrypt password hashing, Helmet security headers, an anti-account-enumeration pattern on password reset, and hashed one-time reset codes. However, the review found **one critical** and **one high-severity** issue that should be treated as an active incident, plus a number of medium/low issues that meaningfully weaken the app's security posture, particularly around **secrets management**, **authentication hardening**, and **operational visibility**.

Most urgent finding: live production credentials — a MongoDB connection string with an embedded database password, the JWT signing secret, a Gmail application password, and third-party API keys — are stored in plaintext in backend/.env. Git history analysis confirms this file was committed to the repository in the past (commit 018cd64, later removed in 7eba670), and the repository has a GitHub remote configured (github.com/umarsoomro1/pocket-trainer.git). Deleting a file from the working tree does **not** remove it from Git history — anyone with clone access to that history (or who obtained a copy before deletion) still has these credentials today unless they have since been rotated. This should be treated as a presumed credential compromise until every listed secret is rotated.

This report frames findings and recommendations using the **NIST Cybersecurity Framework (CSF) 2.0** functions — Govern, Identify, Protect, Detect, Respond, and Recover — to help prioritize not just code fixes but the surrounding process (secret rotation practice, monitoring, incident response) needed to keep the app secure going forward.

2. Scope & Methodology

In scope:

- Mobile client (Expo / React Native): screens, navigation, auth context, Axios API client, push notifications.
- Backend API (Node.js / Express, deployed on Vercel serverless functions): auth, workouts, chat/AI routes.
- Data layer: Mongoose models (User, WorkoutPlan, ChatMessage, DailyLog) on MongoDB Atlas.
- Third-party integrations: Modal-hosted AI model endpoint, RapidAPI ExerciseDB, Gmail SMTP (Nodemailer), Google GenAI / Resend packages present in dependencies.
- Configuration and secrets handling: backend/.env, vercel.json, .gitignore, and Git history relevant to those files.

Method: manual, static source-code review of the supplied project archive (no dynamic/penetration testing was performed against the live Vercel deployment or MongoDB Atlas cluster, and no attempt was made to exploit or exfiltrate the exposed credentials beyond confirming their presence in the codebase and Git history).

Out of scope: mobile OS-level hardening (jailbreak/root detection), infrastructure configuration of the Vercel project and MongoDB Atlas network/IP access rules, and the internals of the third-party Modal AI model.

3. Findings Register (Severity Overview)

Ten security findings and one supply-chain/hygiene observation were identified, summarized below and detailed in Section 4.

#	Finding	Severity	Primary NIST CSF 2.0 Mapping
F1	Live secrets committed to Git history / stored in plaintext .env	CRITICAL	PR.AA-05, GV.SC-05
F2	No rate limiting or lockout on authentication endpoints	HIGH	PR.AA-03, DE.CM-03
F3	Weak JWT secret handling: insecure fallback + 30-day non-revocable tokens	MEDIUM	PR.AA-01, PR.AA-05
F4	Overly permissive CORS policy (all origins allowed)	MEDIUM	PR.PS-05
F5	Auth token stored unencrypted in AsyncStorage on the mobile client	MEDIUM	PR.DS-01
F6	No password strength/complexity enforcement	MEDIUM	PR.AA-01
F7	Unvalidated/unbounded user input forwarded into AI prompts	MEDIUM	PR.PS-01, ID.RA-01
F8	Raw internal error messages returned to API clients	LOW	PR.PS-01
F9	No structured logging, monitoring, or alerting	LOW	DE.CM-01, DE.AE-02
F10	No documented incident response / secret-rotation runbook	MEDIUM	RS.MA-01, RC.RP-01
F11	Full Git history, node_modules, and logs shipped in delivered archive	INFO	GV.SC-05, ID.AM-01

4. Detailed Findings

F1. Live secrets committed to Git history / stored in plaintext .env

CRITICAL

NIST CSF 2.0 mapping: PROTECT — PR.AA-05 (credentials and secrets managed), PR.DS-01 (data-at-rest protection); GOVERN — GV.SC-05 (supply-chain / secrets policy); IDENTIFY — ID.AM-01 (asset inventory should include secrets)

Impact: backend/.env contains a full MongoDB Atlas connection string with an embedded database username/password, the application's JWT signing secret, a Gmail application password used for outbound email, and API keys for the AI model and the RapidAPI ExercisedB service. Anyone who obtains this file (or this repository's history) gains: full read/write access to the production database (every user's account, weight/health data, and chat history), the ability to mint valid JWTs for any user ID (full account takeover), the ability to send email as the app's Gmail account, and free use of paid third-party API quotas billed to the project owner.

Root cause, reconstructed from Git history: .gitignore correctly excluded .env and backend/.env as of commit 61d3ac6 (Aug 18, 02:39). Roughly 16.5 hours later, commit 4329c6e — authored by a second contributor on this repo (MrAG96) — removed both lines from .gitignore while updating the SDK, with no indication this was intentional. That left the working tree unprotected for just over a week. During that window, commit 018cd64 (Aug 25, 19:51) added backend/.env — with the real secrets already in it — to the repository. It was deleted from the tree 14 minutes later in commit 7eba670 (Aug 26, 00:05), and the .gitignore protection was only restored 18 minutes after that, in commit 1aa2cdc (Aug 26, 00:23). By then the secrets were already permanently in history. This is a process gap, not a one-off typo: a single contributor's .gitignore edit was enough to remove the safety net for everyone working on the repo.

Prior audit was not followed through on: the repository already contains a self-generated security_audit_report.pdf/.docx, added in commit 65a50ec ("Security Audit", Sep 1) and updated in 597403d ("Added security recommendations in the log", Sep 2) — both after the secret had already been committed and removed. Whatever that review covered, it did not result in the leaked credentials being rotated or purged from history by the time of this review, which is itself a gap in the incident-response process (see F10).

Evidence:

```
backend/.env (delivered file) –  
MONGO_URI=mongodb+srv://soomrou40_db_user:<redacted>@pocket-trainer.u6h32kp.mongodb.net/...
```

```
backend/.env – JWT_SECRET, AI_API_KEY, RAPIDAPI_KEY, EMAIL_USER / EMAIL_PASS (Gmail app password)  
also present in plaintext
```

```
git show 61d3ac6 -- .gitignore → Aug 18 02:39, umarsoomro1 adds ".env" / "backend/.env" to  
.gitignore
```

```
git show 4329c6e -- .gitignore → Aug 18 19:05, MrAG96 removes those same two lines ("SDK Updated")
```

```
git log --all --diff-filter=A -- backend/.env → 018cd64 "AI integrated to backend" (Aug 25 19:51)  
adds the file while unprotected
```

```
git log --all --oneline -- backend/.env → 7eba670 "Delete backend/.env" (Aug 26 00:05) removes it  
from the tree only – history still retains it
```

```
git show 1aa2cdc -- .gitignore → Aug 26 00:23, ".env" / "backend/.env" re-added – 8 days, 5 hours  
after protection was first dropped
```

```
git remote -v → origin https://github.com/umarsoomro1/pocket-trainer.git
```

Recommendations:

- Treat this as an active incident: rotate every credential immediately — MongoDB Atlas database user password, JWT_SECRET, the Gmail app password, the AI/Modal API key, and the RapidAPI key.
- After rotating, purge the secret from Git history (e.g. `git filter-repo` or BFG Repo-Cleaner), force-push, and have any collaborators re-clone.
- If the GitHub remote repository is or was ever public, assume the secrets are already known to third parties and audit MongoDB Atlas access logs / billing on the AI and RapidAPI accounts for unauthorized use.
- Move all secrets out of committed files entirely: use Vercel's encrypted Environment Variables for production and a local, git-ignored `.env` for development only, and confirm `.env` is (and stays) in `.gitignore` before the first commit of any new secret.
- Protect `.gitignore` itself: require review on any pull request that touches it, or add a pre-commit/CI secret-scanning hook (e.g. `gitleaks` or `truffleHog`) that blocks a push containing an `.env` file regardless of what `.gitignore` currently says — so a single accidental edit can't silently reopen this exposure again.
- Consider a dedicated secrets manager (Vercel env vars, Doppler, AWS/GCP Secrets Manager) with scoped access and audit logging for anything beyond a small side project.

F2. No rate limiting or lockout on authentication endpoints

HIGH

NIST CSF 2.0 mapping: PROTECT — PR.AA-03 (users/devices authenticated commensurate with risk); DETECT — DE.CM-03 (personnel/technology activity monitored for anomalies)

Impact: `/api/auth/login`, `/register`, `/forgot-password`, and `/reset-password` accept unlimited requests with no throttling, CAPTCHA, or account lockout. Login and password endpoints can be brute-forced at whatever rate the attacker's network allows. The forgot-password flow issues a 6-digit numeric OTP (only 900,000 possible values) valid for 15 minutes with no attempt cap on `/reset-password`, making the OTP itself brute-forceable well within its validity window from a single low-cost client.

Evidence:

`backend/server.js` — no rate-limiting middleware (e.g. `express-rate-limit`) is registered globally or per-route.

`backend/routes/authRoutes.js` — `/login`, `/register`, `/forgot-password`, `/reset-password` have no throttling middleware.

`backend/controllers/authController.js` — `resetPassword()` checks the hashed OTP and expiry but does not cap the number of guesses.

Recommendations:

- Add IP- and account-based rate limiting (`express-rate-limit` or a WAF/edge rule on Vercel) to all authentication routes, with stricter limits on `/forgot-password` and `/reset-password`.
- Add a progressive lockout or CAPTCHA challenge after repeated failed login/OTP attempts.
- Track and cap OTP verification attempts server-side (e.g. invalidate the code after 5 failed tries) independent of its 15-minute expiry.

F3. Weak JWT secret handling: insecure fallback + long-lived, non-revocable tokens

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.AA-01 (identities/credentials managed), PR.AA-05 (access permissions managed)

Impact: generateToken() falls back to the hardcoded string 'secretkey' if JWT_SECRET is ever unset (e.g. a misconfigured deployment environment), which would let anyone forge valid tokens. Separately, tokens are issued with a 30-day expiry and there is no server-side revocation list or refresh-token rotation: a token issued before a password change, logout, or compromise remains valid for up to 30 days with no way to invalidate it early.

Evidence:

```
backend/controllers/authController.js – jwt.sign({ id }, process.env.JWT_SECRET || 'secretkey', { expiresIn: '30d' })
```

```
backend/controllers/authController.js – updatePassword() rotates the password hash but issues no new token and revokes no old one.
```

Recommendations:

- Remove the 'secretkey' fallback entirely — fail fast (refuse to start) if JWT_SECRET is not set.
- Shorten access-token lifetime (e.g. 15–60 minutes) and introduce refresh tokens with server-side revocation (a token/jti blacklist or a stored token version per user).
- Invalidate existing sessions when a password is changed or reset (e.g. bump a per-user tokenVersion claim checked on every request).

F4. Overly permissive CORS policy

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.PS-05 (configuration management follows secure baselines)

Impact: The Express app calls cors() with no configuration, which reflects and allows requests from any origin. For a public fitness app the practical risk is lower than for an internal tool, but combined with the bearer-token auth model it still widens the attack surface for browser-based credential misuse against any future web client, and offers no defense-in-depth if a frontend origin is ever compromised.

Evidence:

```
backend/server.js – app.use(cors()); // no origin allowlist configured
```

Recommendations:

- Restrict CORS to the known frontend origin(s) (the Expo web build's domain, and any admin/web app) via an explicit allowlist.
- If a public web client is not currently planned, consider disabling browser CORS entirely and relying on the mobile app's direct API calls.

F5. Auth token stored unencrypted on the mobile client

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.DS-01 (protection of data-at-rest)

Impact: The JWT is persisted via @react-native-async-storage/async-storage, which is unencrypted on-device storage. On a rooted/jailbroken device, or via a malicious app with storage access, the token can be read directly and replayed to impersonate the user for up to its full 30-day lifetime.

Evidence:

```
api/axiosConfig.js and context/AuthContext.js – AsyncStorage.setItem('userToken', ...) / getItem('userToken')
```

Recommendations:

- Store the auth token in platform secure storage (expo-secure-store, backed by iOS Keychain / Android Keystore) instead of AsyncStorage.
- Combine with the shorter-lived token strategy from F3 so any leaked token has a smaller exposure window.

F6. No password strength enforcement

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.AA-01 (identities and credentials are managed)

Impact: Registration and password-reset flows only check that a password field is non-empty; there is no minimum length or complexity rule on either the RegisterScreen (client) or registerUser/resetPassword (server). Weak, short, or common passwords are accepted, increasing exposure to credential-stuffing and offline cracking if the password hash database is ever obtained.

Evidence:

screens/RegisterScreen.jsx — only checks `!email || !password || !dob || !weight`, no length/format rule.

backend/controllers/authController.js — registerUser()/resetPassword() only check the fields are present.

Recommendations:

- Enforce a minimum length (e.g. 10+ characters) and reject common/breached passwords (e.g. via a k-anonymity check against the Have I Been Pwned Pwned Passwords API) on both client and server.
- The project already depends on zod — use it to define and enforce a shared validation schema for all auth request bodies.

F7. Unvalidated, unbounded user input forwarded into AI prompts

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.PS-01 (secure software development practices); IDENTIFY — ID.RA-01 (risk assessment covers third-party/AI dependencies)

Impact: `sendMessage()` takes the raw chat message and interpolates it directly into a prompt sent to the Modal AI endpoint, with no length limit, content filtering, or prompt-injection safeguards beyond a light regex used only to decide whether the message looks like a workout-modification request. A crafted message could attempt to override the system prompt's instructions, and there is no cap on message size, so a very long message could also drive up cost/latency against the metered AI backend or exhaust the 75-second request timeout budget.

Evidence:

```
backend/controllers/chatController.js — sendMessage() passes req.body.message (trimmed only)
straight into generateChatResponse()/modifyWorkoutPlan().
```

```
backend/services/aiService.js — prompt = `...User Message: "${message}"` with no length cap or
escaping before being sent to MODAL_AI_URL.
```

Recommendations:

- Cap chat message length server-side and reject/truncate oversized input before it reaches the AI service.
- Keep the existing `sanitizeModifiedSession()` muscle-group guardrails (a good mitigation already in place) and extend similar allow-list validation to any other AI-influenced writes to the database.
- Add basic abuse/rate limiting on the chat endpoint specifically, since each call incurs a billed third-party AI request.

F8. Raw internal error messages returned to API clients

LOW

NIST CSF 2.0 mapping: PROTECT — PR.PS-01 (secure development / secure defaults)

Impact: Nearly every controller returns `{ message: error.message }` on failure, which can leak internal details (Mongoose validation internals, driver error text, occasionally file paths) to any API caller, aiding reconnaissance of the backend's internals.

Evidence:

```
backend/controllers/*.js — repeated pattern: catch (error) { res.status(500).json({ message:
error.message }); }
```

Recommendations:

- Add a centralized error-handling middleware that logs the full error internally but returns a generic message to the client in production, reserving detailed messages for development mode only.

F9. No structured logging, monitoring, or alerting

LOW

NIST CSF 2.0 mapping: DETECT — DE.CM-01 (networks/systems monitored), DE.AE-02 (adverse events analyzed)

Impact: Observability is limited to scattered `console.log/console.error` calls. There is no centralized log aggregation, no alerting on authentication anomalies (e.g. spikes in failed logins, which would also help catch the brute-force risk in F2), and no application performance/error monitoring, which will slow detection and investigation of any real incident — including whether the F1 credentials have already been misused.

Recommendations:

- Adopt structured logging (e.g. pino/winston) with request IDs, and ship logs to a persistent store rather than serverless function stdout only.

- Add an error-tracking/APM tool (e.g. Sentry) and basic alerting for spikes in 401/429 responses or database connection failures.

F10. No documented incident response or secret-rotation runbook

MEDIUM

NIST CSF 2.0 mapping: RESPOND — RS.MA-01 (incident response plan executed); RECOVER — RC.RP-01 (recovery plan executed)

Impact: This is no longer a theoretical gap. Git history shows a prior self-review already existed in this repository (security_audit_report.pdf/.docx, committed Sep 1 and updated Sep 2) — added after the F1 secret had already been committed and removed from the tree — yet the credentials were still live and unrotated at the time of this review. That confirms there is no working process for acting on a known-or-suspected credential leak once it's identified, which matters more than whether the leak is discovered.

Evidence:

```
git show --stat 65a50ec → "Security Audit" (Sep 1): adds security_audit_report.pdf/.docx
```

```
git show --stat 597403d → "Added security recommendations in the log" (Sep 2): updates the same report
```

```
backend/.env (delivered file) → the F1 secrets are still present, live, and unrotated as of this review
```

Recommendations:

- Write a short runbook covering: who to notify, how to rotate each class of secret (DB, JWT, email, third-party API keys), how to force-invalidate sessions, and how to check for signs of misuse in MongoDB Atlas / provider logs.
- Treat any finding from a prior review as open until there is a verifiable action (a rotated key, a merged fix) — not just a written report — closing it.
- Periodically rehearse or at least review the runbook as the app and its dependencies grow.

F11. Full Git history, node_modules, and local logs shipped in the delivered archive

INFO

NIST CSF 2.0 mapping: GOVERN — GV.SC-05 (supply chain / handling of sensitive artifacts); IDENTIFY — ID.AM-01 (asset inventory)

Impact: The archive provided for this review included the full .git directory (which is how the F1 history was recoverable), node_modules for both the app and backend, and Expo log/cache files, totaling roughly 500MB. This is an operational hygiene point rather than a code vulnerability: any time this project is zipped up and shared (for review, backup, or handoff), the exposure surface for F1-style history leakage grows.

Recommendations:

- When sharing the project for review or handoff, export a clean copy (e.g. `git archive` from a fresh clone, or exclude .git/, node_modules/, and .expo/) unless the recipient specifically needs full history.
- This reinforces the priority of purging and rotating the F1 secrets rather than relying on the deleted file staying unnoticed in history.

5. NIST CSF 2.0 Function Alignment

The table below gives a qualitative snapshot of how the current codebase and (as far as observable from source) its surrounding process align to each NIST CSF 2.0 function. This is a directional assessment based on a source-code review, not a formal maturity certification.

Function	Snapshot	Posture	Key gaps observed
GOVERN (GV)	No visible secrets policy or supply-chain handling standard; no documented data-classification for user health/fitness data.	Weak	F1, F10, F11
IDENTIFY (ID)	Data model is simple and understood, but there is no formal inventory of secrets, third-party data flows (AI vendor, RapidAPI, Gmail), or associated risk.	Partial	F1, F7
PROTECT (PR)	Solid basics (bcrypt, Helmet, hashed OTPs) undermined by secrets exposure, no rate limiting, permissive CORS, and unencrypted mobile token storage.	Partial / Weak	F1–F7
DETECT (DE)	Only ad-hoc console logging; no monitoring for authentication abuse, error spikes, or anomalous API traffic.	Weak	F2, F9
RESPOND (RS)	No documented incident response process; the F1 secret exposure appears to have been handled by file deletion only.	Weak	F10
RECOVER (RC)	MongoDB Atlas presumably has its own backup tooling, but no application-level recovery/rotation plan is documented.	Not evidenced	F10

6. Prioritized Remediation Roadmap

Immediate

0 – 7 days

- Rotate MongoDB Atlas database credentials, JWT_SECRET, Gmail app password, AI/Modal API key, and RapidAPI key (F1).
- Purge backend/.env from Git history and force-push; confirm .gitignore covers it going forward (F1, F11).
- Remove the 'secretkey' JWT fallback so the app fails to start without a real secret (F3).
- Add rate limiting to /login, /register, /forgot-password, /reset-password (F2).

Short-term

2 – 4 weeks

- Introduce short-lived access tokens + refresh-token rotation with server-side revocation (F3).
- Move mobile token storage from AsyncStorage to expo-secure-store (F5).
- Restrict CORS to known frontend origins (F4).
- Enforce password strength rules using the existing zod dependency, on client and server (F6).
- Cap chat message length and add input validation across auth/workout/chat request bodies (F6, F7).
- Add a centralized error handler that hides internal error detail in production (F8).

Ongoing / Longer-term

1 – 3 months

- Adopt a managed secrets store (Vercel encrypted env vars, Doppler, or a cloud secrets manager) (F1).
- Add structured logging + an error-tracking/APM tool, with alerting on auth anomalies (F9).
- Write and circulate an incident-response / secret-rotation runbook (F10).
- Add automated dependency vulnerability scanning (npm audit / Dependabot / Snyk) to CI.
- Periodically review MongoDB Atlas network access rules and API key scopes/quotas for least privilege.

7. Recommendations Quick-Reference

A one-line recommendation for every finding, for scanning at a glance. Full context, evidence, and rationale for each is in Section 4.

#	Recommendation	Severity
F1	Rotate every credential in backend/.env now; purge it from Git history; move secrets to a managed secrets store.	CRITICAL
F2	Add rate limiting/lockout to login, register, forgot-password, and reset-password.	HIGH
F3	Remove the JWT fallback secret; shorten token lifetime and add server-side revocation.	MEDIUM
F4	Restrict CORS to known, explicit frontend origins.	MEDIUM
F5	Store the auth token in expo-secure-store instead of AsyncStorage.	MEDIUM
F6	Enforce a minimum password length/strength on client and server (zod schema).	MEDIUM
F7	Cap chat message length and validate/sanitize input before it reaches the AI service.	MEDIUM
F8	Return generic error messages to clients in production; log full detail server-side only.	LOW
F9	Add structured logging plus an error-tracking/APM tool with basic alerting.	LOW
F10	Write a short incident-response / secret-rotation runbook; treat past findings as open until verifiably fixed.	MEDIUM
F11	Share a clean export (no .git/node_modules) when handing off or reviewing the project.	INFO

8. Positive Observations

The following existing practices are good and should be preserved as the recommendations above are implemented:

- Passwords are hashed with bcrypt (bcryptjs, salt rounds 10) rather than stored or compared in plaintext.
- Helmet is applied globally for baseline HTTP security headers.
- The forgot-password flow returns an identical response whether or not the email exists, preventing account enumeration.
- Password-reset codes are hashed (SHA-256) before being persisted, so the database never holds the raw OTP.
- User-scoped endpoints (workout plan, dashboard, chat) derive the target record from the authenticated req.user rather than from client-supplied IDs, avoiding an obvious insecure-direct-object-reference (IDOR) pattern.
- The AI-driven workout modifier includes a sanitizeModifiedSession() guardrail that filters clearly mismatched exercises (e.g. blocking shoulder/triceps moves on a pure chest day) before writing AI output back to the database.

9. Additional Recommendations to Improve the App

Beyond the security findings above, the following engineering and product-quality improvements would make PocketTrainer more robust, maintainable, and trustworthy — several of them also reduce security risk indirectly (e.g. schema validation, environment separation).

Data validation & schema hardening

- Add Mongoose-level constraints (enum for goal, min/max for weight, string length caps) so bad data can't reach the database even if a controller-level check is missed — the zod dependency already in package.json is a good fit for this at the request layer too.
- Validate email format server-side (not just presence) on register/login/forgot-password.

Testing & CI/CD

- No automated tests were found in the reviewed source (package.json's test script is a placeholder). Add unit tests for controllers/services (especially auth and the AI-response sanitization logic) and basic integration tests for the API routes.
- Add a CI pipeline (GitHub Actions) that runs tests, linting, and dependency audit on every push/PR before deploying to Vercel.

Environment & configuration hygiene

- Separate development, staging, and production environments with their own database, JWT secret, and API keys — right now there's a single shared configuration.
- server.js mounts every route under both /api/x and /x 'so requests never 404' — this is a workaround rather than a fix; align the mobile client and Vercel rewrites on one canonical path and remove the duplicate mounting.

Code quality & maintainability

- aiService.js currently hardcodes the entire exercise/workout-plan library in source; consider moving it to the database or a config file so it can be updated without a redeploy, and to reduce duplication with exerciseService.js's exercise-name matching.
- Consider adopting TypeScript (or at least JSDoc types) for the backend, given the growing surface of controllers/services and the risk of shape mismatches between the AI service's JSON output and the Mongoose schemas.
- Centralize the repeated try/catch + `res.status(500).json({ message: error.message })` pattern (see F8) into shared middleware to reduce duplication as well as fix the information-disclosure issue.

Product & UX resilience

- getExerciseGif() only matches a small hardcoded dictionary of exercise names; AI-generated or user-modified exercise names that don't match will silently show no demonstration GIF — consider a fallback (generic muscle-group image, or a real exercise-database API lookup) instead of null.
- The chat feature has no pagination on getChatHistory() — history will be fetched in full on every load, which will slow down over time for active users. Add pagination or a reasonable history limit.
- Consider a lightweight in-app disclaimer that AI-generated fitness/nutrition guidance is not medical advice, given the system prompt asks the model for 'medically sound guidance' and exact macro/nutrition breakdowns.

Data privacy considerations

- The app stores date of birth, weight history, and fitness goals — treat this as sensitive personal data: document a retention/deletion policy (e.g. what happens to this data on account deletion, which isn't currently implemented) and consider whether applicable privacy regulations (e.g. GDPR/CCPA depending on your user base) require a formal policy and user-facing controls.

10. Appendix — Files Reviewed

Backend (Node.js / Express):

- server.js, backend/api/index.js, backend/config/db.js, backend/vercel.json
- backend/controllers/authController.js, workoutController.js, chatController.js
- backend/middleware/authMiddleware.js
- backend/models/User.js, WorkoutPlan.js, ChatMessage.js, DailyLog.js
- backend/routes/authRoutes.js, workoutRoutes.js, chatRoutes.js
- backend/services/aiService.js, emailService.js, exerciseService.js
- backend/.env, backend/package.json

Mobile client (Expo / React Native):

- App.jsx, index.js, theme.js, metro.config.js, app.json, eas.json, package.json
- api/axiosConfig.js, context/AuthContext.js, navigation/AppNavigation.jsx
- services/notificationService.js
- screens/LoginScreen.jsx, RegisterScreen.jsx, DashboardScreen.jsx, ActiveWorkoutScreen.jsx, ChatScreen.jsx, ProfileScreen.jsx, ProgressSummaryScreen.jsx, AboutScreen.jsx

Repository metadata:

- Git commit history and remote configuration (analyzed to confirm the F1 secret-exposure timeline).
- .gitignore

This report reflects a manual source-code review only. It does not constitute a penetration test, does not verify current MongoDB Atlas network/IAM configuration, and does not confirm whether the exposed credentials in Finding F1 have been misused — that requires reviewing provider-side access logs, which was outside the scope of this review.